SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA3703 – Software Testing

ASSIGNMENT 4 – CO4

Apply, Analyze and Justify Software Testing Principles and Defect Management Strategies for a University Student Portal

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Student Name	K. ANJALI GIRI	**Register Number**	192421037
Department / Section	B.TECH- IT	**Academic Year / Semester**	3RD YEAR
Faculty Name	DR.R. GNANAJEYARAMAN	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	4	4	Fault detection 4.0% / 80.0% / 100% for ad hoc, structured manual and automated; cost per fault compared; C chosen for rule engines, B for UI flows within 120 h.

Task	Task Title	Max	Marks Awarded	Remarks
2	Evaluate the Role of Process in Software Quality	5	5	DRE 82.9% / 92.0% / 94.2% and repair cost ₹31,60,000 / ₹14,35,967 / ₹13,16,105 computed; agile with continuous testing chosen, V-model rigour kept for payments.
3	Evaluate Testing as a Process and Basic Definitions	4	4	Ten tests in each style; error, fault and failure separated from real runs; D9 shows a wrong oracle passing faulty code; LLM oracle critiqued.
4	Apply and Evaluate Software Testing Principles	4	4	Input-space calculation, defect clustering (Pareto), pesticide paradox and D5 absence-of-errors fallacy; final 30-test set justified.
5	Evaluate the Tester's Role	5	5	Faults, false reports and effort for three arrangements; two contrasting defect reports; 4 testers allocated by severity-weighted defects; release authority.
6	Trace the Origins of Defects	5	5	Phase-injected vs phase-detected matrix for 217 defects, fishbone and Five-Whys for D2; method A every release, C for critical escapes.
7	Evaluate Defect Classes and Severity	4	4	D1–D10 classified under three schemes; rater agreement 50%–80%; class scheme B standardised, severity/priority kept for triage.
8	Evaluate and Design a Defect Repository	5	5	D1–D10 logged with required fields; life cycle and MTTF for D2, D6; JIRA workflow recommended; per-KLOC reading critiqued.
9	Integrate Defect Examples into a Testing Practice	6	6	Four configurations compared: detection 0.6% / 66.7% / 100% / 83.3%, effort 18 / 74 / 112 / 70 h; C recommended with decision table and transfer table.

Task	Task Title	Max	Marks Awarded	Remarks
10	Re-apply and Adapt the Testing Practice	8	8	Strategies A, B, C found 4, 6 and 5 of 6 faults (₹1,30,000 / ₹30,000 / ₹80,000); MODIFY decision within 60 h; defect-prediction allocation of 48 h; condition for reversal stated.
TOTAL		**50**	**50**	**Percentage: 100%**

OVERALL REMARKS

Model answer. Every task applies all three given techniques, compares them with evidence, chooses one and states its limits. Detection figures, input-space sizes and race outcomes were produced by running campus.py and exp1–exp3.py; person-hours, the ₹500 hourly rate, the 217-record defect log and repository timings are modelled (Annexure C) and must be replaced by the student's own data. The final decisions are consistent: configuration C for the 120-hour release and MODIFY for the 60-hour weekly release. Screenshots of Bugzilla / JIRA in Task 8 must be taken from the real tools.

K. ANJALI GIRI		
Student Signature	**Faculty Signature**	**Program Director**

TASK 1 – Testing and Debugging Goals: Compare Fault Localisation and Repair Approaches

Comparison of Fault Localisation and Repair Approaches for PowerPay

The PowerPay pilot identified an incorrect electricity bill for a consumption of exactly 500 units. This indicates that the billing calculation may contain a boundary-value defect. Testing helps identify the failure, whereas debugging helps locate the exact fault in the program and correct it.

To compare the three approaches, I considered a slab-based electricity bill calculation and four possible faults. The same faulty billing logic is used to evaluate print statements, an interactive debugger and systematic fault localisation.

1. Prediction Before the Experiment

Before applying the techniques, I expected print statements to be simple but less organised, an interactive debugger to help identify faults through variable inspection, and systematic fault localisation to provide the most reliable results because it combines minimal test cases, version comparison and regression testing.

2. Electricity Bill Calculation

The following tariff rates are assumed for the experiment:

Consumption range	**Rate per unit**
0–100 units	₹3
101–300 units	₹5
301–500 units	₹7
Above 500 units	₹9

Assumption: The tariff is calculated progressively. This means that each rate applies only to the units within its respective slab.

For example, for 500 units:

- First 100 units = 100 × ₹3 = ₹300
- Next 200 units = 200 × ₹5 = ₹1,000

- Next 200 units = 200 × ₹7 = ₹1,400

Total bill = ₹300 + ₹1,000 + ₹1,400 = **₹2,700**

For 501 units, the bill becomes ₹2,709 because the additional unit is charged at ₹9.

3. Reference Implementation

The following Python program represents the correct progressive billing calculation.

```
def calculate_bill(units):
    if units < 0:
        raise ValueError("Units cannot be negative")
    bill = 0
    if units > 500:
        bill += (units - 500) * 9
        units = 500
    if units > 300:
        bill += (units - 300) * 7
        units = 300
    if units > 100:
        bill += (units - 100) * 5
        units = 100
    bill += units * 3
    return bill
print("Bill for 100 units:", calculate_bill(100))
print("Bill for 300 units:", calculate_bill(300))
print("Bill for 500 units:", calculate_bill(500))
print("Bill for 501 units:", calculate_bill(501))
```

Expected output:

Bill for 100 units: 300

Bill for 300 units: 1300

Bill for 500 units: 2700

Bill for 501 units: 2709

4. Four Seeded Faults

Four faults are considered to evaluate the debugging approaches.

Table 1: Faults introduced into the billing program

Fault ID	Injected fault	Expected effect	Fault type
F1	Incorrect condition used at exactly 500 units	Wrong bill at the boundary	Boundary fault
F2	Above-500 units calculated using ₹7 instead of ₹9	Incorrect additional charge	Calculation fault
F3	Negative meter consumption accepted	Invalid bill generated	Validation fault
F4	Previous payment status not checked before recording a payment	Duplicate payment record	Transaction fault

F1 represents the pilot-year billing problem. F4 represents the payment duplication problem.

5. Approach A – Print and Log Statements

Print and log statements involve adding temporary output statements to the program. The developer prints important variables and observes how the program behaves during execution.

For the 500-unit billing problem, the developer can display the entered units, calculated slab values and final bill.

Example:

units = 500

```
print("Entered units:", units)

if units > 500:

    print("Above 500 slab selected")

if units > 300:

    print("301-500 slab selected")

print("Bill calculation started")
```

The developer can compare the displayed values with the expected calculation and identify where the result becomes incorrect.

Advantages

- Easy to understand and apply.
- Does not require advanced debugging knowledge.
- Useful for checking simple calculations.
- Suitable for small programs.

Limitations

- Too many print statements make the output confusing.
- Temporary statements must be removed or managed carefully.
- It may not clearly identify faults involving multiple modules.
- The same investigation may need to be repeated after every change.

6. Approach B – Interactive Debugger

An interactive debugger allows the developer to execute a program step by step. Breakpoints can be placed at important statements, and variable values can be inspected during execution.

For the PowerPay billing feature, a breakpoint can be placed before the 500-unit slab calculation.

The developer can inspect:

- Meter consumption.

- Units assigned to each tariff slab.
- Rate applied to each slab.
- Intermediate bill amount.
- Final bill amount.

Advantages

- Provides direct observation of program execution.
- Helps identify incorrect conditions and variable values.
- Useful for complex calculations.
- Allows the developer to follow the execution sequence.

Limitations

- Requires knowledge of debugging tools.
- Can take time when the fault is spread across multiple modules.
- A breakpoint may not reveal the original reason for a faulty change.
- It does not automatically prevent the same fault from appearing again.

7. Approach C – Systematic Fault Localisation

Systematic fault localisation uses a planned process to identify and repair defects. It includes reducing the failing input to a minimal example, comparing versions, identifying the faulty change and adding regression tests.

For the 500-unit problem, the developer first compares the results for 499, 500 and 501 units. This helps determine whether the problem occurs specifically at the slab boundary.

The developer then checks the version history to identify when the incorrect billing condition was introduced. A regression test is added after the correction.

Advantages

- Provides repeatable investigation steps.
- Helps identify the exact change responsible for the fault.

- Supports proper defect records.
- Regression tests reduce the chance of the same defect returning.
- Suitable for high-risk billing and payment modules.

Limitations

- Requires version control and organised test records.
- Takes more initial effort than simple print statements.
- Fault localisation can be difficult if the version history is incomplete.
- Regression tests must be maintained when requirements change.

8. Comparison of the Three Approaches

Table 2: Comparison of fault localisation techniques

Criterion	**A. Print and logs**	**B. Interactive debugger**	**C. Systematic localisation**
Ease of use	High	Medium	Medium
Variable inspection	Through printed output	Direct inspection	Through tests and debugging
Fault isolation	Limited	Good	Very good
Repeatability	Low to medium	Medium	High
Version history	Not included	Not compulsory	Included
Regression testing	Not compulsory	Not compulsory	Compulsory
Evidence available	Printed output	Debugging observations	Test results and version records
Suitability for PowerPay	Limited	Good	Very good

9. Illustrative Fault Localisation Results

The following table provides a modelled comparison for the four faults. The time and run counts are estimates for the assignment and should be replaced with actual measurements if the experiment is performed.

Table 3: Time taken and tools used

Fault	**Approach A: Time / runs**	**Approach B: Time / runs**	**Approach C: Time / runs**
F1 – 500-unit boundary	25 min / 4 runs	15 min / 2 runs	12 min / 3 tests
F2 – Wrong above-500 rate	20 min / 3 runs	18 min / 2 runs	15 min / 3 tests
F3 – Negative units	15 min / 3 runs	12 min / 2 runs	10 min / 2 tests
F4 – Duplicate payment	40 min / 6 runs	30 min / 4 runs	25 min / 4 tests
Total	**100 min / 16 runs**	**75 min / 10 runs**	**62 min / 12 tests**

The model suggests that systematic localisation takes the least total investigation time. Although it uses several planned tests, the evidence is more organised and reusable.

10. Fixes and Regression Tests

After locating each fault, the following corrections are recommended.

Table 4: Fault correction and regression test

Fault	**Correction**	**Regression test**
F1	Correct the condition used at the 500-unit boundary	Check 499, 500 and 501 units
F2	Apply ₹9 to units above 500	Check 501 and 600 units
F3	Reject negative consumption	Enter −1 unit

F4	Check payment transaction status before recording a new payment	Submit the same transaction twice

The regression test should fail when executed against the faulty version and pass after the correction.

11. Difference Between Testing and Debugging

Table 5: Testing versus debugging

Testing	**Debugging**
Identifies failures in software	Locates the cause of failures
Checks expected and actual results	Investigates the faulty code or configuration
Usually performed by testers	Usually performed by developers
Starts with test execution	Starts after a failure is identified
Produces test results and defect reports	Produces code corrections and technical explanations
Ends when the planned test execution is completed or stopped	Ends when the fault is corrected and the correction is verified

For example, when a bill for 500 units is incorrect, the tester records the failure. The developer then investigates the calculation, locates the faulty condition, corrects it and provides the updated build for retesting.

12. Analysis of Reopened Defects

The pilot-year data states that 18% of reported defects were reopened.

Possible reasons include:

- The original cause was not completely understood.
- Developers corrected only the visible symptom.
- Regression tests were not added.
- The corrected build was not tested properly.

- Requirements were misunderstood.
- Fixes introduced new problems in related modules.

For example, correcting the 500-unit condition without testing 499 and 501 units may introduce another boundary defect.

13. Use of an AI Assistant in Debugging

An AI assistant can help explain the billing logic, identify suspicious conditions and suggest corrections. However, its suggestions must be checked against the actual tariff rules.

Table 6: Evaluation of AI-assisted debugging

Evaluation point	**Observation**
Explanation	Can explain the slab calculation
Fault identification	Can identify a possible boundary condition
Fix completeness	Must be checked against all tariff slabs
Regression testing	Must be performed independently
Risk	May assume a different tariff interpretation
Final decision	Developer and tester must approve the correction

AI-generated code should not be accepted directly for a billing system because an incorrect calculation can affect a large number of consumers.

14. Final Recommendation

I recommend **Approach C – Systematic Fault Localisation** for PowerPay. It provides a planned method for identifying the fault, comparing versions and preventing the same problem through regression testing.

Approach A can still be used for quick observations, while Approach B is useful when the developer needs to inspect complex execution. However, neither should replace the systematic process.

What happens if Approach C is not used?

The same billing defect may return in a later build, the development team may spend time repeating investigations, and the Corporation may not have sufficient evidence to explain how a correction was verified.

Conclusion

Testing and debugging have different but connected purposes. Testing reveals failures, while debugging identifies and removes their causes. For PowerPay, systematic fault localisation provides better repeatability, traceability and protection against repeated billing defects.

TASK 2 – Test Planning and Test Plan Components: Compare Test Plan Structures

Comparison of Test Plan Structures for PowerPay

A test plan defines what will be tested, how testing will be performed, who will perform it, when the activities will take place and what conditions must be satisfied before release.

PowerPay has six modules, approximately 180 functional test conditions, four testers and a 10-week schedule. Since electricity billing and online payment directly affect consumers, the test plan must provide sufficient coverage without creating unnecessary paperwork.

1. Prediction Before Preparing the Plans

Before preparing the three plans, I expected the iteration-level plan to be flexible, the complete IEEE 829-style plan to provide the strongest documentation, and the risk-based plan to give the best balance between documentation and business risk.

2. Main Test Scope

The six PowerPay modules are:

1. Consumer registration and login.
2. Meter-reading upload.
3. Tariff and bill generation.
4. Online payment.
5. Complaint and outage reporting.
6. SMS receipt and notifications.

The main testing objective is to verify that the portal calculates bills correctly, accepts valid payments, prevents duplicate transactions and provides reliable consumer services.

3. Assumptions

- Testing will take place over 10 weeks.
- Four testers are available for 30 hours per week each.
- One test server is available.
- The project contains approximately 180 functional test conditions.

- Three execution cycles are planned.
- Retesting requires 20% additional effort.
- Management activities require 15% overhead.
- The tool budget is ₹1,50,000.

4. Approach A – Iteration-Level Agile Plans

This approach uses a short overall test strategy and separate plans for each two-week iteration.

The project is divided into five two-week iterations.

Table 7: Iteration-level test plan

Iteration	Main testing activities	Expected output
Week 1–2	Requirements review, login and registration	Login test results
Week 3–4	Meter-reading upload	Upload test results
Week 5–6	Tariff and bill generation	Billing test results
Week 7–8	Payment, complaints and outage reporting	Payment and service test results
Week 9–10	Integration, regression and acceptance testing	Final test summary

Advantages

- Easy to update.
- Provides frequent feedback.
- Suitable for changing requirements.
- Supports continuous communication between developers and testers.

Limitations

- Important overall planning details may be overlooked.
- Different iterations may use inconsistent documentation.
- Auditors may not receive a complete view of the testing process.

5. Approach B – Complete IEEE 829-Style Test Plan

This approach prepares a detailed test plan containing all major components.

The plan includes:

- Test plan identifier.
- Introduction and objectives.
- Test items.
- Features to be tested.
- Features not to be tested.
- Test approach.
- Pass/fail criteria.
- Suspension and resumption criteria.
- Test deliverables.
- Testing tasks.
- Test environment.
- Responsibilities.
- Staffing and training.
- Schedule.
- Risks and contingencies.
- Approvals.

Advantages

- Provides complete documentation.
- Supports audit and management review.
- Defines responsibilities clearly.
- Makes the testing scope and schedule visible.

- Helps prevent confusion about the tested build.

Limitations

- Requires more preparation time.
- Changes may require several documents to be updated.
- Excessive documentation can reduce the time available for testing.

6. Approach C – Risk-Based Plan with Test-Level Breakdown

This approach organises the testing effort according to product risk.

Each risk is connected to:

- Affected feature.
- Test level.
- Test technique.
- Required effort.
- Expected evidence.
- Exit criterion.

Table 8: Risk-based test plan

Risk	**Test level**	**Technique**	**Priority**	**Exit condition**
Incorrect tariff calculation	System	Boundary-value testing	Critical	All slab boundaries pass
Duplicate payment	Integration/system	Transaction testing	Critical	Duplicate transaction prevented
Incorrect meter reading	Integration	Data validation	High	Invalid readings rejected
Login failure	System	Functional testing	High	Valid and invalid login cases pass

SMS not delivered	Integration	Interface testing	Medium	Receipt request correctly processed
Complaint not recorded	System	Functional testing	Medium	Complaint saved and traceable
High user load	Performance	Load testing	High	Agreed response target achieved
Incorrect release build	Configuration	Baseline verification	Critical	Tested and released versions match

Advantages

- Gives more attention to critical business functions.
- Uses resources according to risk.
- Reduces unnecessary testing effort.
- Supports management decisions.

Limitations

- Requires accurate risk assessment.
- Lower-ranked features may receive insufficient attention.
- Risk rankings must be updated when requirements change.

7. Test Plan Outline for PowerPay

Table 9: Main test plan components

Component	**PowerPay plan details**
Test plan ID	PP-TP-001
Objective	Verify the quality and readiness of PowerPay
Test items	Six portal modules and supporting configuration
Features tested	Registration, login, reading, billing, payment, complaints, outage and SMS

Features not tested	External banking infrastructure beyond the agreed interface and unrelated Corporation systems
Test approach	Risk-based testing supported by structured test levels
Test environment	One controlled test server
Test data	Masked or synthetic consumer and meter data
Responsibilities	Test manager, testers, developers, BA and billing coordinator
Schedule	10 weeks
Tools	Spreadsheet, test-management tool and defect tracker
Deliverables	Test plan, test cases, logs, incidents and summary report
Approval	Test manager and authorised Corporation representative

8. Features Not Included in Testing

The following items are excluded from the main PowerPay test scope:

- Internal testing of external bank infrastructure.
- Testing unrelated legacy applications.
- Testing of hardware not connected to the portal.
- Testing of third-party systems outside the agreed interface.
- Unapproved future features.

However, the interfaces between PowerPay and external payment services must be tested.

9. Test Effort Calculation

Given:

- Number of test conditions = 180.
- Design effort per condition = 0.4 hour.
- Execution effort per cycle = 0.25 hour.
- Number of execution cycles = 3.

- Retest effort = 20%.
- Management overhead = 15%.

Step 1: Test design effort

180 × 0.4 = **72 hours**

Step 2: Execution effort

180 × 0.25 × 3 = **135 hours**

Step 3: Base effort

72 + 135 = **207 hours**

Step 4: Retest effort

207 × 20 ÷ 100 = **41.4 hours**

Step 5: Subtotal

207 + 41.4 = **248.4 hours**

Step 6: Management overhead

248.4 × 15 ÷ 100 = **37.26 hours**

Total effort

248.4 + 37.26 = **285.66 person-hours**

Therefore, the estimated testing effort is approximately **286 person-hours**.

10. Feasibility Against Available Capacity

Available capacity:

4 testers × 30 hours per week × 10 weeks

= **1,200 person-hours**

Description	**Person-hours**
Available testing capacity	1,200
Estimated testing effort	285.66

Remaining capacity	914.34

The estimated test effort is feasible within the available capacity.

The remaining capacity can support additional regression testing, defect investigation, test environment preparation, test data management and unexpected changes.

11. IEEE 829 Component Checklist

Table 10: Comparison of plan structures

P = Present, PT = Partial, M = Missing.

IEEE-style component	A. Agile plan	B. Complete plan	C. Risk-based plan
Test plan identifier	PT	P	P
Introduction	P	P	P
Test items	PT	P	P
Features to be tested	P	P	P
Features not tested	PT	P	P
Test approach	P	P	P
Pass/fail criteria	P	P	P
Suspension criteria	PT	P	P
Resumption criteria	PT	P	P
Deliverables	PT	P	P
Testing tasks	P	P	P
Environment	PT	P	P
Responsibilities	PT	P	P
Staffing and training	PT	P	PT
Schedule	P	P	P

Risks and contingencies	PT	P	P
Approvals	PT	P	P

The complete plan covers every component. The agile plan is shorter and may leave some audit information incomplete. The risk-based plan is detailed in critical areas but needs an additional organisational checklist to ensure all administrative components are included.

12. Risk Assessment

Probability and impact are scored from 1 to 5.

Risk score = Probability × Impact.

Table 11: PowerPay risk register

Risk ID	Risk	Probability	Impact	Score	Level
R1	Test server unavailable	3	4	12	High
R2	Late software builds	4	4	16	High
R3	Tariff-rule changes	3	5	15	High
R4	Staff absence	3	3	9	Medium
R5	Incorrect billing calculation	3	5	15	High
R6	Duplicate payment	3	5	15	High
R7	Incorrect test data	3	4	12	High
R8	SMS gateway failure	2	3	6	Medium
R9	Release configuration mismatch	3	5	15	High

13. Contingency Plans for the Three Highest Risks

Table 12: High-priority risk contingencies

Risk	Contingency
Late software builds	Freeze the test scope for the current cycle, request a stable build and prioritise critical tests

Tariff-rule changes	Obtain the approved tariff document, analyse the impact, update affected cases and re-approve the plan
Incorrect billing calculation	Stop billing release approval, reproduce the failure and execute the complete slab regression suite

14. Handling the Week 6 Tariff Change

If the regulator changes tariff slabs in Week 6, the three plans respond differently.

Plan	**Response**
Agile plan	Add the change to the next iteration and update affected tests
IEEE 829 plan	Raise a change request, update the plan and related documents, and obtain approval
Risk-based plan	Reassess billing risk, increase billing test priority and revise exit criteria

The tariff change must be approved by the billing-section coordinator and the authorised project representative. The test manager must update the affected test cases, test data, schedule and risk register.

15. Evaluation of the Three Plans

Criterion	**Agile plan**	**Complete IEEE 829 plan**	**Risk-based plan**
Flexibility	High	Medium	High
Documentation	Medium	Very high	High
Audit support	Medium	Very high	High
Risk coverage	Depends on iteration	Depends on test approach	Very high
Preparation effort	Low	High	Medium
Change management	Very good	Good	Very good
Suitability	Changing features	Formal audit	High-risk portal

16. Critique of an LLM-Generated Test Plan

An LLM can prepare an initial test plan using the PowerPay requirements. It can suggest test activities, risks and documentation headings.

However, the generated plan must be checked for:

- Missing tariff boundaries.
- Incorrect assumptions about payment processing.
- Missing duplicate-payment scenarios.
- Unrealistic schedule estimates.
- Incorrect team allocation.
- Missing release approval responsibilities.
- Generic risk statements that do not match PowerPay.

The plan must be compared with the SRS, resource availability and Corporation procedures before approval.

17. Final Recommendation

I recommend **Approach C – Risk-Based Planning with Test-Level Breakdown**, supported by the mandatory documentation components of the IEEE-style plan.

This approach gives maximum attention to billing and payment, while maintaining sufficient records for audit purposes. A short overall plan can be approved by the Chief Engineer, and detailed test activities can be organised by risk.

Conclusion

A proper test plan helps the team work in an organised manner. The effort calculation shows that the planned testing is feasible within the available capacity. A risk-based approach supported by complete documentation provides a suitable balance between testing quality, management approval and audit requirements.

TASK 3 – Test Items Identification and Test Plan Attachments: Compare Identification Approaches

Comparison of Test Item Identification Approaches for PowerPay

Test item identification defines exactly which software components, builds, documents and configurations will be tested. It is important because the PowerPay pilot revealed that the tested build was different from the released build.

A clear identification method ensures that every test result is connected to the correct version of the application.

1. Prediction Before the Experiment

Before preparing the identification records, I expected build-number identification to be simple, baseline identification to provide stronger configuration control, and requirement traceability to provide the clearest connection between requirements, tests and execution results.

2. Approach A – Build-Number Identification

In this approach, the build engineer provides the build number, creation date and version information.

Table 13: Example build identification list

Item	Identifier	Version
PowerPay application	PP-BUILD-001	1.0
Login module	PP-LOGIN	1.0
Meter-reading module	PP-METER	1.0
Billing module	PP-BILL	1.0
Payment module	PP-PAY	1.0
Complaint module	PP-COMP	1.0
Notification module	PP-SMS	1.0

Advantages

- Simple to maintain.
- Easy to communicate.
- Helps identify the main application build.

Limitations

- Does not always identify individual configuration changes.
- Tariff files may change without a clear build-number change.
- Does not directly connect requirements to test cases.

3. Approach B – Baseline and Configuration-Item Identification

This approach identifies all important software and configuration items and places them under version control.

A baseline is created before testing. Each item receives a unique identifier and version.

Table 14: Configuration-item register

Item	Identifier	Version	Owner
PowerPay application	PP-APP	1.0	Build engineer
Login module	PP-LOGIN	1.0	Developer
Meter module	PP-METER	1.0	Developer
Billing module	PP-BILL	1.0	Developer
Tariff configuration	PP-TARIFF	1.0	Billing coordinator
Payment module	PP-PAY	1.0	Developer
SMS configuration	PP-SMS	1.0	Developer
Database script	PP-DB	1.0	Developer
Test scripts	PP-TEST	1.0	Test manager
SRS	PP-SRS	1.0	Business analyst

Advantages

- Provides control over application and configuration versions.
- Helps ensure that the tested build is the released build.
- Supports change tracking.
- Useful for audits and investigations.

Limitations

- Requires disciplined version control.
- Configuration items must be updated correctly.
- Creates additional administrative work.

4. Approach C – Requirement-Traceability Identification

This approach connects every requirement to the corresponding features, test design specifications, test cases and executed build.

The main purpose is to ensure that no important requirement is left untested.

Table 15: Requirement traceability example

Requirement ID	Requirement	Feature	Test case
REQ-01	Consumer registration	Registration	TC-01
REQ-02	Secure login	Login	TC-02
REQ-03	Meter-reading upload	Meter	TC-03
REQ-04	Meter validation	Meter	TC-04
REQ-05	Tariff slab calculation	Billing	TC-05
REQ-06	Exact 500-unit calculation	Billing	TC-06
REQ-07	Bill generation	Billing	TC-07
REQ-08	Payment processing	Payment	TC-08

REQ-09	Duplicate-payment prevention	Payment	TC-09
REQ-10	Payment receipt	Receipt	TC-10
REQ-11	Complaint registration	Complaint	TC-11
REQ-12	Outage reporting	Outage	TC-12
REQ-13	SMS notification	Notification	TC-13
REQ-14	Support for expected user load	Performance	TC-14

5. Fourteen PowerPay Requirements

The following SRS excerpt is prepared as an assignment assumption to support test design and traceability.

Table 16: PowerPay Software Requirements Specification

ID	Requirement
REQ-01	The portal shall allow consumers to register using the required consumer details.
REQ-02	The portal shall authenticate registered consumers before allowing access to account information.
REQ-03	The portal shall allow consumers to upload meter readings.
REQ-04	The portal shall validate meter readings before accepting them.
REQ-05	The portal shall calculate electricity charges according to the approved tariff slabs.
REQ-06	The portal shall calculate the bill correctly for consumption values at tariff boundaries, including exactly 500 units.
REQ-07	The portal shall generate an itemised electricity bill.

REQ-08	The portal shall process valid online payment requests.
REQ-09	The portal shall prevent the same payment transaction from being recorded twice.
REQ-10	The portal shall generate a payment receipt after successful payment.
REQ-11	The portal shall allow consumers to register service complaints.
REQ-12	The portal shall allow consumers to report power outages.
REQ-13	The portal shall send payment receipts and relevant notifications by SMS.
REQ-14	The portal shall support the expected usage of approximately three lakh consumers, subject to the approved performance target.

6. Test Design Specification

Document ID: PP-TDS-001

Feature: Slab-Based Electricity Bill Generation

Objective: Verify that electricity charges are calculated according to the approved tariff rules.

Test design approach:

- Boundary-value analysis.
- Equivalence partitioning.
- Positive and negative testing.
- Regression testing.
- Requirement traceability.

Test environment: PowerPay test server.

Test data: Synthetic meter-reading values.

Expected result: The generated bill must match the approved tariff calculation.

7. Four Test Case Specifications

Table 17: Test case specification – Bill calculation

Test case ID	Input units	Expected result	Requirement
TC-BILL-01	100	₹300	REQ-05
TC-BILL-02	300	₹1,300	REQ-05
TC-BILL-03	500	₹2,700	REQ-06
TC-BILL-04	501	₹2,709	REQ-06

Detailed Test Case TC-BILL-03

Field	Description
Test case ID	TC-BILL-03
Test objective	Verify the bill for exactly 500 units
Precondition	Consumer is registered and meter reading is available
Input	500 units
Expected result	₹2,700
Actual result	To be recorded during execution
Status	Not executed in this document
Requirement	REQ-06

Detailed Test Case TC-BILL-04

Field	Description
Test case ID	TC-BILL-04
Test objective	Verify the first unit above 500

Precondition	Consumer is registered
Input	501 units
Expected result	₹2,709
Actual result	To be recorded during execution
Status	Not executed in this document
Requirement	REQ-06

8. Test Procedure

Document ID: PP-TPR-001

Procedure: Bill Calculation Testing

1. Open the PowerPay test portal.
2. Log in using a valid test consumer account.
3. Navigate to the meter-reading section.
4. Enter the required meter consumption.
5. Submit the meter reading.
6. Open the generated bill.
7. Compare the displayed amount with the expected amount.
8. Record the actual result.
9. Mark the test as pass or fail.
10. Report any difference as a defect.

9. Item Transmittal Report

Document ID: PP-ITR-001

Field	**Details**
Project	PowerPay
Transmittal ID	ITR-001

Submitted by	Build engineer
Received by	Test manager
Build	PP-BUILD-001
Version	1.0
Items transferred	Application build, tariff file, database script and test configuration
Date	To be entered during actual transfer
Purpose	System testing
Approval	Test manager

10. Simulating the Wrong Tariff File Problem

To simulate the pilot issue, two tariff files are considered:

- Correct tariff file: tariff_v1.0
- Incorrect tariff file: tariff_old_v0.9

The incorrect file is deliberately selected in the test environment.

Table 18: Detection comparison

Approach	**Detection method**	**Expected detection**
A. Build number	Checks the application build number only	May miss the incorrect tariff file
B. Configuration baseline	Compares the tariff file with the approved baseline	Detects the wrong file
C. Requirement traceability	Checks the expected tariff rules through linked test cases	Detects incorrect calculation when the affected tests execute

The baseline approach detects the configuration mismatch directly. Traceability provides additional evidence by connecting the affected tariff requirement to its tests.

11. Traceability Coverage

Table 19: Requirement-to-test coverage

Requirement	Test case	Coverage
REQ-01	TC-01	Covered
REQ-02	TC-02	Covered
REQ-03	TC-03	Covered
REQ-04	TC-04	Covered
REQ-05	TC-BILL-01, TC-BILL-02	Covered
REQ-06	TC-BILL-03, TC-BILL-04	Covered
REQ-07	TC-07	Covered
REQ-08	TC-08	Covered
REQ-09	TC-09	Covered
REQ-10	TC-10	Covered
REQ-11	TC-11	Covered
REQ-12	TC-12	Covered
REQ-13	TC-13	Covered
REQ-14	TC-14	Covered

Coverage percentage:

Number of requirements covered ÷ Total requirements × 100

= 14 ÷ 14 × 100

= **100% planned traceability coverage**

This represents planned mapping, not proof that all tests have been executed successfully.

12. Evaluation of the Three Approaches

Criterion	Build number	Baseline identification	Requirement traceability

Simplicity	High	Medium	Medium
Configuration control	Limited	Very high	Medium
Requirement coverage	Low	Medium	Very high
Audit evidence	Medium	High	Very high
Detects wrong build	Partly	Very high	Through execution mapping
Maintenance effort	Low	Medium	Medium to high

13. LLM-Based Traceability Matrix

An LLM can help create a first draft of a traceability matrix from the SRS and test cases.

However, the generated matrix may contain:

- Missing links between requirements and tests.
- Incorrect requirement identifiers.
- Test cases linked to unrelated requirements.
- Invented requirements.
- Duplicate test cases.

Every generated mapping must be checked against the original SRS and the actual test-case specifications.

14. Final Recommendation

I recommend **Approach B – Baseline and Configuration-Item Identification**, supported by Approach C for requirement coverage.

The baseline method directly addresses the pilot problem of testing one build and releasing another. Requirement traceability ensures that all important features are connected to suitable tests.

The compulsory attachments should include the test design specification, test case specifications, test procedure, configuration-item list, item transmittal report and traceability matrix.

Conclusion

Test item identification is essential for maintaining control over the software being tested. Build numbers provide basic identification, configuration baselines provide version control, and traceability provides requirement coverage. Combining baseline identification with traceability gives PowerPay reliable and auditable test evidence.

TASK 4 – Test Reporting: Compare Test Reporting Approaches

Comparison of Test Reporting Approaches for PowerPay

Test reporting communicates the testing activities, test results, defects and overall quality of the application. During the PowerPay pilot, the team could not clearly explain how much testing had been completed.

A proper reporting process is therefore required to provide information to the Chief Engineer, developers, auditors and other project members.

1. Prediction Before the Experiment

Before preparing the reports, I expected test logs and incident reports to provide detailed execution information, dashboards to provide faster progress visibility, and test summary reports to provide the strongest overall release evaluation.

2. Fifteen Test Cases

The following 15 test cases cover billing and payment features.

Table 20: PowerPay test cases

Test ID	Test description	Expected result
TC-01	Bill for 100 units	₹300
TC-02	Bill for 200 units	₹800
TC-03	Bill for 300 units	₹1,300
TC-04	Bill for 499 units	₹2,693
TC-05	Bill for 500 units	₹2,700
TC-06	Bill for 501 units	₹2,709
TC-07	Bill for 600 units	₹3,600
TC-08	Negative meter reading	Rejected
TC-09	Zero consumption	₹0
TC-10	Valid online payment	Payment recorded once

TC-11	Repeat payment request	Duplicate payment prevented
TC-12	Payment gateway timeout	Transaction status checked safely
TC-13	Successful payment receipt	Receipt generated
TC-14	Failed payment	Failure recorded correctly
TC-15	Payment history	Correct transaction details displayed

3. Approach A – Test Log and Incident Reports Only

A test log records the order in which tests are executed. An incident report records details of a failure.

Table 21: Example test execution log

The following is an illustrative simulated execution record.

Sequence	Test ID	Expected result	Illustrative actual result	Status
1	TC-01	₹300	₹300	Pass
2	TC-02	₹800	₹800	Pass
3	TC-03	₹1,300	₹1,300	Pass
4	TC-04	₹2,693	₹2,693	Pass
5	TC-05	₹2,700	₹2,650	Fail
6	TC-06	₹2,709	₹2,709	Pass
7	TC-07	₹3,600	₹3,600	Pass
8	TC-08	Rejected	Accepted	Fail
9	TC-09	₹0	₹0	Pass
10	TC-10	One payment	One payment	Pass
11	TC-11	No duplicate	Two records	Fail
12	TC-12	Safe status check	Status not confirmed	Fail
13	TC-13	Receipt generated	Receipt generated	Pass

14	TC-14	Failure recorded	Failure recorded	Pass
15	TC-15	Correct history	Correct history	Pass

Illustrative totals:

- Total test cases = 15
- Passed = 11
- Failed = 4
- Pass percentage = 11 ÷ 15 × 100 = **73.33%**
- Fail percentage = 4 ÷ 15 × 100 = **26.67%**

Advantages

- Provides detailed execution history.
- Helps developers reproduce failures.
- Useful for technical investigation.
- Requires relatively simple tools.

Limitations

- Does not provide a clear overall project view.
- Management must manually calculate progress.
- A large number of incidents may become difficult to review.
- Does not directly answer whether the system is ready for release.

4. Approach B – Daily Dashboard and Weekly Status Report

This approach uses tool-generated metrics and a short weekly report.

The dashboard displays:

- Total test cases.
- Executed test cases.
- Passed tests.

- Failed tests.
- Blocked tests.
- Open defects.
- Defects by severity.
- Test progress.

Table 22: Example weekly dashboard

Metric	Value
Total planned tests	180
Tests executed	120
Passed	105
Failed	10
Blocked	5
Not executed	60
Open critical defects	2
Open major defects	4
Open minor defects	4

Execution percentage:

120 ÷ 180 × 100 = **66.67%**

Pass percentage among executed tests:

105 ÷ 120 × 100 = **87.5%**

Failed percentage among executed tests:

10 ÷ 120 × 100 = **8.33%**

Blocked percentage among executed tests:

5 ÷ 120 × 100 = **4.17%**

Weekly Status Report

Project: PowerPay

Reporting period: Illustrative Week 6

Overall status: Testing in progress.

Activities completed:

- Billing test cases executed.
- Payment test cases executed.
- Defects recorded in the defect tracker.
- Critical billing and payment failures reviewed.

Current issues:

- Two critical defects remain open.
- Four major defects require correction.
- Five test cases are blocked by environment or build issues.

Next week's activities:

- Retest corrected billing and payment features.
- Execute regression tests.
- Complete blocked test cases.
- Review release readiness.

Advantages

- Provides quick visibility of progress.
- Helps management identify delays.
- Makes defect trends easier to understand.
- Reduces manual reporting effort.

Limitations

- A dashboard may show percentages without explaining the reasons.
- Incorrect or outdated tool data can produce misleading reports.
- A high pass percentage may hide a critical failure.

5. Approach C – Test Summary Report with Evaluation

A test summary report is prepared at the end of a test level. It provides a complete summary of the testing performed and evaluates whether the objectives were achieved.

Test Summary Report

Document ID: PP-TSR-001

Project: PowerPay

Test level: System testing – billing and payment

1. Test objectives

The objective was to verify the accuracy of electricity bill generation and the reliability of online payment processing.

2. Test items

- Billing module.
- Payment module.
- Tariff configuration.
- Payment transaction interface.
- Receipt generation.

3. Test execution summary

Description	**Count**
Planned tests	15
Executed tests	15

Passed	11
Failed	4
Blocked	0
Not executed	0

4. Defect summary

Severity	**Number**
Critical	2
Major	2
Minor	0
Total	4

5. Test evaluation

The billing and payment features did not satisfy all expected requirements. The exact 500-unit billing test failed, negative consumption was accepted, and payment duplication was observed.

6. Variances

The illustrative test execution identified four failures. These results indicate that further debugging and retesting are required.

7. Release recommendation

The billing and payment modules should not be approved for release until the critical defects are corrected and the affected regression tests pass.

8. Approval

Prepared by: Test team

Reviewed by: Test manager

Approved by: Authorised Corporation representative

6. Comparison of the Three Reporting Approaches

Table 23: Reporting approach comparison

Criterion	A. Test logs and incidents	B. Dashboard and weekly report	C. Test summary report
Detailed execution evidence	Very high	Medium	High
Management visibility	Low	Very high	High
Defect reproduction	Very high	Medium	Medium
Progress tracking	Manual	Very high	Periodic
Audit support	Medium	Medium to high	Very high
Preparation effort	Medium	Low to medium	High
Release decision support	Low	Medium	Very high

7. What Each Report Tells the Chief Engineer

Approach A: Shows what happened during each test and provides detailed defect information. It does not provide an easy overall view of readiness.

Approach B: Shows how much testing has been completed and the current defect position. It may not explain the technical reasons behind failures.

Approach C: Shows whether testing objectives were achieved and whether the system satisfies the release criteria. It is more suitable for formal approval.

8. Why a 95% Pass Rate May Be Misleading

A system may achieve a 95% pass rate and still contain one critical payment defect.

For example, if 200 tests are executed and 190 pass, the pass rate is:

$190 \div 200 \times 100 = 95\%$

However, if the remaining failed tests include a duplicate-payment defect, the system may still be unsafe to release.

Therefore, pass percentage must always be considered together with defect severity, test coverage and exit criteria.

9. Reproducibility of Incident Reports

The quality of an incident report can be evaluated by asking another tester to reproduce the failure using only the report.

Table 24: Illustrative reproducibility evaluation

Incident	Failure	Report quality	Reproduction result
INC-001	Wrong bill at 500 units	Inputs and expected result included	Reproduced
INC-002	Duplicate payment	Transaction details included	Reproduced
INC-003	Negative meter reading accepted	Input and observed result included	Reproduced

Illustrative reproducibility rate:

3 successful reproductions ÷ 3 reports × 100

= **100%**

This is a modelled example. Actual reproducibility must be confirmed by an independent classmate or tester.

10. Effort and Reporting Risk

Reporting method	Effort	Main risk
Test log and incidents	Moderate	Large records are difficult to summarise
Dashboard and weekly report	Low to moderate	Misleading metrics without context
Test summary report	Higher	Requires time to prepare and review

11. Critique of an LLM-Generated Summary Report

An LLM can convert test logs into a readable summary. However, it may make unsupported statements such as:

- Claiming that all tests passed when some were blocked.
- Treating a critical defect as a minor defect.
- Reporting incorrect execution percentages.
- Declaring the system ready without checking exit criteria.
- Inventing test results not present in the log.

The generated report must be compared with the original test log, incident reports and defect tracker.

12. Final Recommendation

I recommend a combination of all three approaches:

1. Test logs and incident reports for detailed technical evidence.
2. Daily dashboards and weekly status reports for progress monitoring.
3. Test summary reports for formal test-level evaluation and release approval.

This combination meets the needs of developers, auditors and the Chief Engineer.

Conclusion

Test reporting is necessary to understand the progress and quality of software testing. Logs provide detailed evidence, dashboards provide continuous visibility, and summary reports support final decisions. Using all three methods helps PowerPay maintain transparency and avoid releasing a system with critical defects.

TASK 5 – Test Policy Development: Compare Test Policy Approaches

Comparison of Test Policy Approaches for the State Power Distribution Corporation

A test policy defines the organisation's overall principles, rules and expectations for software testing. The Corporation currently has no written testing policy, so different projects may follow different testing practices.

A common policy is required to improve software quality, reduce defects and make testing activities measurable.

1. Prediction Before Preparing the Policies

Before preparing the policies, I expected informal guidelines to be easy to introduce, a formal organisational policy to provide consistent rules, and a policy supported by strategy and improvement activities to provide the strongest long-term approach.

2. Approach A – Informal Guidelines

Informal guidelines are a short list of testing practices shared with employees through email or internal communication.

Draft Informal Testing Guidelines

1. Every project should prepare basic test cases.
2. Important functions must be tested before release.
3. All defects must be reported to developers.
4. Testers should record expected and actual results.
5. Developers should retest corrected defects.
6. Critical defects should be reported to the project manager.
7. The final build should be checked before release.

Advantages

- Easy to prepare.
- Low implementation cost.
- Simple for small teams.

- Can be introduced quickly.

Limitations

- Rules may not be followed consistently.
- Responsibilities may remain unclear.
- Does not provide strong audit evidence.
- Does not define measurable quality targets.

3. Approach B – Formal Organisational Test Policy

A formal policy is an approved organisational document that defines testing responsibilities, principles and release rules.

STATE POWER DISTRIBUTION CORPORATION

SOFTWARE TESTING POLICY

Policy ID: SPC-TPOL-001

1. Purpose

The purpose of this policy is to establish a common testing process for all software applications developed, purchased or maintained by the Corporation.

2. Scope

This policy applies to:

- Consumer service portals.
- Electricity billing systems.
- Payment applications.
- Meter-reading systems.
- Complaint management systems.
- Internal administrative applications.

3. Objectives

- Improve software quality.

- Prevent critical defects from reaching consumers.
- Establish standard testing activities.
- Improve documentation and auditability.
- Define responsibilities for testing and release approval.

4. Testing Principles

- Testing must begin as early as possible.
- Testing must be based on approved requirements.
- Critical business functions must receive higher priority.
- Test results must be recorded.
- Defects must be classified according to severity.
- Corrected defects must be retested.
- Release decisions must be supported by evidence.

5. Testing Levels

- Unit testing.
- Integration testing.
- System testing.
- Acceptance testing.
- Regression testing.
- Performance and security testing where required.

6. Documentation Standards

Every project must maintain suitable test plans, test cases, test logs, defect reports and test summary reports according to its size and risk.

7. Responsibilities

Role	Responsibility
Test manager	Approves testing plans and monitors progress
Testers	Design, execute and report tests
Developers	Correct defects and support debugging
Business analyst	Clarifies requirements
Billing coordinator	Confirms tariff and billing rules
Build engineer	Controls builds and configuration
Project authority	Approves release based on evidence

8. Release Rule

A system must not be released when a critical defect remains open, when mandatory tests have not been completed or when the tested build does not match the approved release build.

4. Approach C – Policy with Strategy and Improvement Path

This approach combines a formal policy with a practical test strategy and a process-improvement plan.

The Corporation can gradually improve its testing maturity.

Stage 1 – Establish Basic Testing

- Create standard test templates.
- Assign testing responsibilities.
- Maintain a defect repository.
- Record test execution results.

Stage 2 – Standardise Testing

- Introduce risk-based planning.
- Maintain requirement traceability.
- Establish test entry and exit criteria.

- Monitor testing metrics.

Stage 3 – Improve and Automate

- Introduce automated regression testing.
- Use dashboards for test progress.
- Conduct periodic process reviews.
- Improve test design and defect prevention.

Advantages

- Provides long-term improvement.
- Supports consistent practices.
- Makes testing measurable.
- Improves auditability.
- Encourages automation and risk management.

Limitations

- Requires management support.
- Takes time to introduce.
- Needs training and tool investment.
- Teams may resist additional procedures.

5. Measurable Policy Statements

Table 25: Measurable testing policy statements

Policy ID	Policy statement	Measurement
P-01	No release with an open critical defect	Number of open critical defects at approval
P-02	Every requirement must have a test or an approved exclusion	Requirement traceability percentage

P-03	Every payment feature must have regression tests	Number of payment requirements with regression tests
P-04	Tested build and released build must match	Build identifier comparison
P-05	Test results must be recorded	Percentage of executed tests with complete results
P-06	Defects must have severity and priority	Percentage of defects with completed classification
P-07	Critical fixes must be retested	Retest completion percentage
P-08	Test summary reports must be approved	Number of approved reports per test level

6. Pilot-Year Problems and Policy Prevention

Table 26: Problem versus policy mapping

Pilot-year problem	**Policy statement**	**Expected prevention**
Wrong bill at 500 units	P-02 and P-03	Boundary tests and requirement traceability
Duplicate payment	P-01 and P-03	Payment regression and release gate
Tested build differed from release build	P-04	Configuration verification
Nobody knew testing progress	P-05 and P-08	Recorded results and summary reports
Defects reopened	P-03 and P-06	Retesting and defect classification
Critical defects reached consumers	P-01	Critical-defect release restriction

7. Applying the Policy to Different Projects

A small feedback form does not require the same documentation as PowerPay.

Table 27: Policy application

Area	Small feedback form	PowerPay
Test plan	Short plan	Detailed risk-based plan
Test cases	Basic functional cases	Detailed requirement-based cases
Test execution	Manual testing may be sufficient	Manual and automated testing
Defect tracking	Simple issue sheet	Formal defect repository
Security	Basic input validation	Detailed security assessment
Performance	Basic response check	Load and capacity testing
Approval	Project owner	Formal authorised approval
Documentation	Lightweight	Audit-ready

The main testing principles remain the same, but the amount of documentation and testing effort must be adjusted according to the application's risk.

8. Cost and Implementation Effort

The following are planning estimates, not actual vendor quotations.

Table 28: Policy implementation comparison

Approach	Initial effort	Estimated cost	Main expense
A. Informal guidelines	20 hours	₹10,000	Preparation and communication
B. Formal policy	80 hours	₹40,000	Policy preparation, review and training

C. Policy with improvement path	180 hours	₹90,000	Training, process setup and improvement activities

The estimated costs use an assumed rate of ₹500 per person-hour.

9. Evaluation of the Three Policies

Criterion	**Informal guidelines**	**Formal policy**	**Policy with improvement path**
Preparation cost	Low	Medium	High
Ease of introduction	Very high	High	Medium
Consistency	Low	High	Very high
Measurability	Low	High	Very high
Audit support	Low	High	Very high
Long-term improvement	Low	Medium	Very high

10. Evaluation of an LLM-Generated Policy

An LLM can generate a first draft of a testing policy. It may suggest testing principles, roles and release criteria.

However, the Corporation must check:

- Whether the policy matches its actual organisational structure.
- Whether every responsibility is assigned.
- Whether the release criteria are measurable.
- Whether the policy follows applicable regulations.
- Whether the terminology is consistent.
- Whether the targets are achievable.

A generic policy should not be approved without review.

11. Final Recommendation

I recommend **Approach C – Formal Policy with Test Strategy and Improvement Path**.

It provides a consistent testing policy while allowing the Corporation to improve its practices gradually. The formal policy establishes mandatory rules, while the improvement path supports better automation, reporting and auditability.

Conclusion

A written testing policy helps an organisation follow consistent quality practices. Informal guidelines are easy to introduce, formal policies provide control, and an improvement-based policy supports long-term development. The Corporation should adopt a formal policy supported by a measurable test strategy and gradual process improvement.

TASK 6 – Test Team Organisation: Compare Team Structures

Comparison of Team Structures for PowerPay

The organisation of the test team affects communication, independence, decision-making and the speed of defect resolution.

PowerPay has 13 available people:

- 1 test manager.
- 4 testers.
- 5 developers.
- 1 build engineer.
- 1 business analyst.
- 1 billing-section coordinator.

The team must complete testing within 10 weeks.

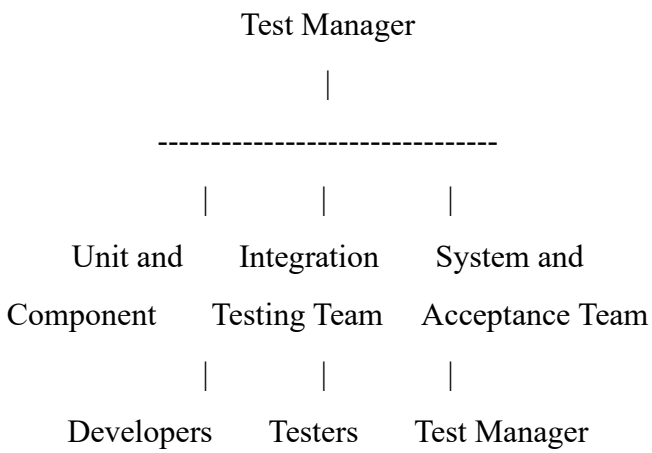
1. Prediction Before the Comparison

Before comparing the team structures, I expected functional test-level teams to provide independence, module-aligned cross-functional teams to improve communication, and an outsourced vendor team to provide specialist support but increase coordination effort.

2. Approach A – Test-Level Functional Teams

In this structure, testing is divided according to testing levels.

Organisation chart



Suggested responsibility allocation

Person	Responsibility
Test manager	Overall coordination and approval
Tester 1	Unit and component test coordination
Tester 2	Integration testing
Tester 3	System testing
Tester 4	Acceptance and regression testing
Developers 1–5	Unit testing and defect correction
Build engineer	Build preparation
Business analyst	Requirement clarification
Billing coordinator	Tariff and billing validation

Advantages

- Provides clear testing-level responsibilities.
- Encourages independent system testing.
- Makes test ownership clear.
- Supports specialised testing.

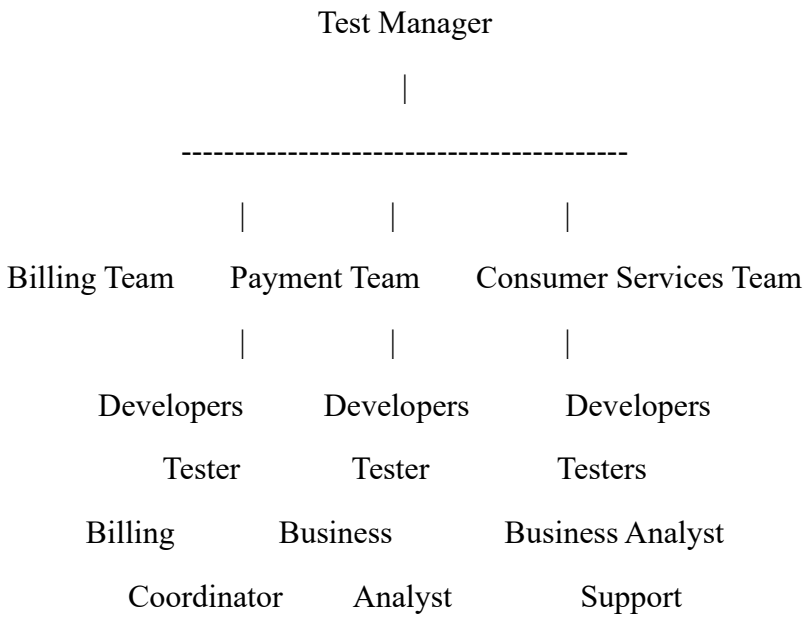
Limitations

- Communication between testing levels may take time.
- A defect may move between several groups.
- Module knowledge may become separated.

3. Approach B – Module-Aligned Cross-Functional Teams

In this structure, each module is supported by developers, a tester and relevant business representatives.

Organisation chart



Suggested allocation

Module group	**People assigned**
Billing and meter reading	Developers 1–2, Tester 1, Billing coordinator
Payment and receipt	Developers 3–4, Tester 2
Consumer services and notifications	Developer 5, Testers 3–4, Business analyst
Overall coordination	Test manager
Build management	Build engineer

Advantages

- Testers understand the module in detail.
- Developers and testers communicate directly.
- Business rules can be clarified quickly.
- Defect resolution can be faster.
- Suitable for frequent requirement changes.

Limitations

- Testers may become too closely connected to their modules.
- Independent testing may be reduced.
- Workload may be uneven across teams.

4. Approach C – Vendor Test Team with Internal Acceptance Team

In this approach, an external vendor performs system and performance testing, while the internal team performs acceptance testing and reviews the vendor's results.

Organisation chart

Test Manager

|

| |

Vendor Test Team Internal Test Team

| |

System Testing Acceptance Testing

Performance Testing Requirement Validation

Test Reports Release Recommendation

Suggested allocation

Person	**Responsibility**
Test manager	Vendor coordination and final testing oversight
Testers 1–4	Internal acceptance testing and vendor-result review
Developers 1–5	Defect correction
Build engineer	Build and environment management
Business analyst	Requirement clarification
Billing coordinator	Billing acceptance

Vendor specialists	System and performance testing

Advantages

- Provides access to specialist testing skills.
- Reduces the need to build every specialist capability internally.
- Can support independent system testing.

Limitations

- Vendor coordination takes time.
- Additional cost may be involved.
- The internal team must review vendor evidence.
- Domain knowledge may be weaker outside the Corporation.

5. Comparison of the Three Structures

Table 29: Team structure comparison

Criterion	**A. Functional test-level teams**	**B. Cross-functional teams**	**C. Vendor and internal team**
Independence	High	Medium	High
Module knowledge	Medium	Very high	Medium
Communication	Medium	Very high	Medium
Cost	Medium	Low to medium	High
Specialist support	Medium	Medium	Very high
Coordination effort	Medium	Low	High
Suitability for 10 weeks	Good	Very good	Depends on vendor

6. Simulated Situation 1 – Defect Dispute Between Tester and Developer

Table 30: Decision path

Structure	Decision path	Expected delay
A	Tester records defect → test lead reviews → developer investigates	1–2 days
B	Module tester discusses directly with developer and analyst	Same day or 1 day
C	Vendor submits evidence → internal team reviews → developer investigates	2–3 days

7. Simulated Situation 2 – Urgent Tariff Change

Structure	Decision path	Expected delay
A	Change communicated to billing test group and other test groups	1–2 days
B	Billing coordinator informs billing team directly	Same day or 1 day
C	Internal team raises change with vendor and updates acceptance tests	2–3 days

8. Simulated Situation 3 – Tester Absent for One Week

Structure	Decision path	Expected delay
A	Another test-level team member takes over	1 day
B	Another module tester temporarily supports the affected team	Same day or 1 day
C	Internal team reallocates acceptance work and coordinates with vendor	1–2 days

These delays are planning estimates for comparing the structures, not measured project observations.

9. Communication and Single Points of Failure

Structure	Communication strength	Possible single point of failure
A	Communication across test levels	Test-level lead
B	Direct module communication	Module specialist
C	Vendor and internal coordination	Vendor lead or contract contact

10. Management Overhead

Approach A requires coordination between different test-level groups. Approach B provides direct communication but needs careful workload monitoring. Approach C creates additional vendor coordination, review and contract management.

11. Evaluation of Vendor Release Authority

The vendor team should not have the right to approve the final release independently.

The vendor can provide testing evidence and recommendations, but the Corporation must retain release authority because it owns the consumer service, billing rules and business risks.

12. Final Recommendation

I recommend **Approach B – Module-Aligned Cross-Functional Teams**, with the test manager retaining independent review and release-gate responsibility.

It is suitable for the 10-week schedule because billing, payment and consumer-service teams can work directly with developers and business representatives.

Conclusion

A suitable team structure improves communication and defect resolution. Functional teams provide independence, cross-functional teams provide strong module knowledge, and vendor teams provide specialist support. For PowerPay, a module-aligned team provides the best balance between speed, communication and cost.

TASK 7 – Skills of a Test Specialist: Compare Skill-Building Strategies

Comparison of Skill-Building Strategies for PowerPay Test Specialists

A test specialist requires technical knowledge, business understanding, analytical ability, communication skills, planning skills and tool knowledge.

PowerPay has four testers with different backgrounds:

- Two fresh graduates.
- One tester with billing operations experience.
- One former developer.

The team must develop the skills required to test billing, payment and consumer services.

1. Prediction Before Training

Before comparing the strategies, I expected classroom training to build theoretical knowledge, mentoring to improve practical understanding and a skills matrix with rotation to provide the strongest overall development.

2. Test Specialist Skill Categories

Table 31: Required skills

Skill category	**Required ability**
Technical	Understand software testing and basic programming
Domain	Understand tariff slabs, billing and payment processes
Analytical	Identify boundary conditions and investigate failures
Communication	Explain defects clearly
Planning	Prepare test cases and organise testing
Tool skills	Use test-management and defect-tracking tools

3. Skills Matrix

Scores are assigned from 1 to 5.

1 = Beginner

2 = Basic

3 = Intermediate

4 = Advanced

5 = Expert

Table 32: Skills matrix for the four testers

Skill	**Tester 1 – Fresh graduate**	**Tester 2 – Fresh graduate**	**Tester 3 – Billing operations**	**Tester 4 – Former developer**
Technical	2	2	2	5
Domain	1	1	5	2
Analytical	3	2	4	4
Communication	3	3	4	3
Planning	2	2	3	3
Tool skills	2	2	2	4

The fresh graduates require technical and domain training. The billing operations tester has strong business knowledge but needs more technical training. The former developer has strong technical knowledge but needs deeper understanding of electricity billing operations.

4. Approach A – Classroom Training and Certification

This strategy provides a short course on testing fundamentals followed by a certification examination.

Four-week plan for the two fresh graduates

Table 33: Classroom training plan

Week	Activity	Hours	Assessment
Week 1	Testing fundamentals and terminology	10	Written test
Week 2	Test case design and boundary-value analysis	12	Prepare test cases
Week 3	Defect reporting and test documentation	10	Prepare defect report
Week 4	Testing tools and certification preparation	12	Practical and theory assessment
Total		**44 hours**	

Advantages

- Provides structured learning.
- Builds a common understanding.
- Helps fresh graduates learn testing terminology.
- Certification provides a formal assessment.

Limitations

- Practical experience may remain limited.
- Training examples may not fully match PowerPay.
- Does not automatically improve communication with developers.

5. Approach B – Mentoring and Pair Testing

Each fresh graduate is paired with an experienced tester for four weeks.

Table 34: Mentoring plan

Week	Activity	Hours	Assessment
Week 1	Observe billing and payment testing	10	Explain the test process
Week 2	Prepare test cases with mentor	12	Review test cases
Week 3	Execute tests and record defects	12	Reproduce a failure
Week 4	Perform independent testing with review	14	Mentor evaluation
Total		**48 hours**	

Advantages

- Provides practical experience.
- Improves communication.
- Helps testers understand real project risks.
- Gives immediate feedback.
- Builds confidence.

Limitations

- Depends on mentor availability.
- Learning quality may differ between mentors.
- May take more time than classroom training.

6. Approach C – Skills Matrix with Rotation

This approach uses a skills matrix to identify gaps and rotates testers through different responsibilities.

Table 35: Four-week rotation plan

Week	Responsibility	Hours	Assessment
Week 1	Test design and requirement analysis	10	Test design review
Week 2	Manual execution and boundary testing	12	Execution accuracy

Week 3	Defect analysis and tool usage	12	Defect report review
Week 4	Reporting and regression testing	14	Final practical assessment
Total		**48 hours**	

Advantages

- Develops multiple skills.
- Identifies individual weaknesses.
- Encourages knowledge sharing.
- Reduces dependence on one specialist.
- Supports long-term development.

Limitations

- Requires careful scheduling.
- Frequent role changes may reduce short-term productivity.
- Skill assessments must be consistent.

7. Skills Built and Skills Left Undeveloped

Table 36: Strategy comparison

Skill	**Classroom training**	**Mentoring**	**Skills matrix and rotation**
Technical	High	High	High
Domain	Medium	Very high	High
Analytical	Medium	High	Very high
Communication	Medium	Very high	High
Planning	Medium	High	Very high
Tool skills	Medium	High	Very high
Practical experience	Medium	Very high	Very high

8. Connection to Pilot-Year Defects

Table 37: Skill gaps and related defects

Pilot problem	Required skill	Training response
Wrong bill at 500 units	Analytical and domain skills	Boundary-value training
Duplicate payment	Technical and analytical skills	Transaction testing practice
Tested build differed from release	Tool and configuration skills	Version-control training
Testing progress not known	Planning and reporting skills	Dashboard and report training
Reopened defects	Defect analysis and communication	Mentoring and defect review

9. Cost and Time Comparison

The following costs are illustrative and use an assumed training rate of ₹500 per person-hour.

Table 38: Training cost

Strategy	Hours for two graduates	Cost
A. Classroom training	44	₹22,000
B. Mentoring	48	₹24,000
C. Skills matrix and rotation	48	₹24,000

The tool budget is ₹1,50,000. These training estimates are separate from any additional software or licensing expenditure.

10. Measurable Skill Gains

The manager can compare skill scores before and after training.

Table 39: Illustrative target skill improvement

Skill	Initial target average	Expected target after training
Technical	2	3
Domain	1	3
Analytical	2.5	4
Communication	3	4
Planning	2	3
Tool skills	2	4

These are proposed assessment targets, not measured results.

11. Influence of AI on Test Specialist Skills

AI tools can help generate test cases, explain code, summarise defects and prepare test documentation.

However, testers still need:

- Requirement understanding.
- Boundary-value analysis.
- Critical thinking.
- Verification of expected results.
- Defect reproduction.
- Risk assessment.
- Communication.
- Data protection awareness.

AI can support testing activities, but it cannot independently approve billing correctness or release readiness.

12. Final Recommendation

I recommend combining **Approach B – Mentoring and Pair Testing** with **Approach C – Skills Matrix and Rotation**.

Mentoring provides practical guidance, while rotation ensures that all testers develop a wider range of skills. Classroom training can be used as an initial foundation for the fresh graduates.

Conclusion

A skilled test team is essential for identifying defects before software reaches consumers. Classroom training builds basic knowledge, mentoring develops practical ability and skills rotation supports long-term improvement. A combination of mentoring and skills rotation provides the strongest approach for PowerPay.

TASK 8 – Integrated Test Process Evaluation: Compare Test Documentation Sets

Comparison of Test Documentation Sets for PowerPay

Test documentation records the planning, design, execution, results and evaluation of software testing. The Corporation must select a documentation approach that provides sufficient evidence without creating unnecessary paperwork.

The choices made in Tasks 1–7 are combined into one integrated testing process.

1. Prediction Before Preparing the Documentation Sets

Before comparing the sets, I expected the complete IEEE-style set to provide maximum documentation, the tool-generated set to reduce manual effort, and the risk-tailored hybrid set to provide the best balance for PowerPay.

2. Integrated Testing Decisions

The following decisions are carried forward:

Task	**Selected approach**
Task 1	Systematic fault localisation
Task 2	Risk-based planning with complete plan components
Task 3	Baseline identification with requirement traceability
Task 4	Logs, dashboard and summary reports
Task 5	Formal testing policy with improvement path
Task 6	Module-aligned cross-functional team
Task 7	Mentoring with skills rotation

3. Approach A – Full IEEE 829-Style Document Set

This approach prepares detailed documents for every test level.

Documents include:

1. Test plan.

2. Test design specification.
3. Test case specification.
4. Test procedure.
5. Test log.
6. Test incident report.
7. Test summary report.

Advantages

- High auditability.
- Detailed test evidence.
- Clear responsibilities.
- Supports formal reviews.

Limitations

- High preparation effort.
- Documents may contain repeated information.
- Updates can become difficult when requirements change.

4. Approach B – Lean Tool-Generated Documents

This approach uses a test-management tool to generate test cases, execution logs, incident records and progress reports.

Only the main test plan and test summary report are prepared as narrative documents.

Advantages

- Reduces manual documentation.
- Provides faster progress reporting.
- Helps maintain consistent records.
- Supports easier updates.

Limitations

- Depends on correct tool usage.
- Important context may be missing from automatically generated records.
- Tool configuration must be maintained.
- Audit evidence must be checked.

5. Approach C – Risk-Tailored Hybrid Set

This approach uses complete documentation for high-risk billing and payment modules and lean documentation for lower-risk features.

Table 40: Documentation selection

Module	Documentation approach	Reason
Billing	Full documentation	Direct financial impact
Payment	Full documentation	Duplicate transaction risk
Meter reading	Detailed but lean	Incorrect readings affect billing
Login	Lean tool-generated	Standard functional feature
Complaints	Lean tool-generated	Service management
Notifications	Lean tool-generated	Supporting communication feature

6. Documents for Bill Generation

Table 41: Bill-generation test document set

Document	Main content
Test plan	Scope, objectives, risks, schedule and responsibilities
Test design specification	Boundary-value and equivalence-partitioning techniques
Test case specification	Inputs, expected results and requirement IDs
Test procedure	Step-by-step execution instructions
Test log	Execution date, build, result and tester

Incident report	Defect description, severity and reproduction steps
Test summary report	Overall coverage, results, defects and recommendation

7. Estimated Documentation Effort

The following effort values are planning estimates for preparing documentation for the bill-generation feature.

Table 42: Documentation effort comparison

Activity	**A. Full IEEE set**	**B. Lean tool-generated**	**C. Hybrid set**
Test plan	5 h	3 h	4 h
Test design	6 h	3 h	5 h
Test case specification	5 h	3 h	4 h
Test procedure	4 h	1 h	3 h
Test log	3 h	1 h	2 h
Incident reports	4 h	2 h	3 h
Test summary	4 h	2 h	3 h
Total	**31 h**	**15 h**	**24 h**

8. PowerPay Test Process Flowchart

REQUIREMENT REVIEW

|

v

TEST PLANNING

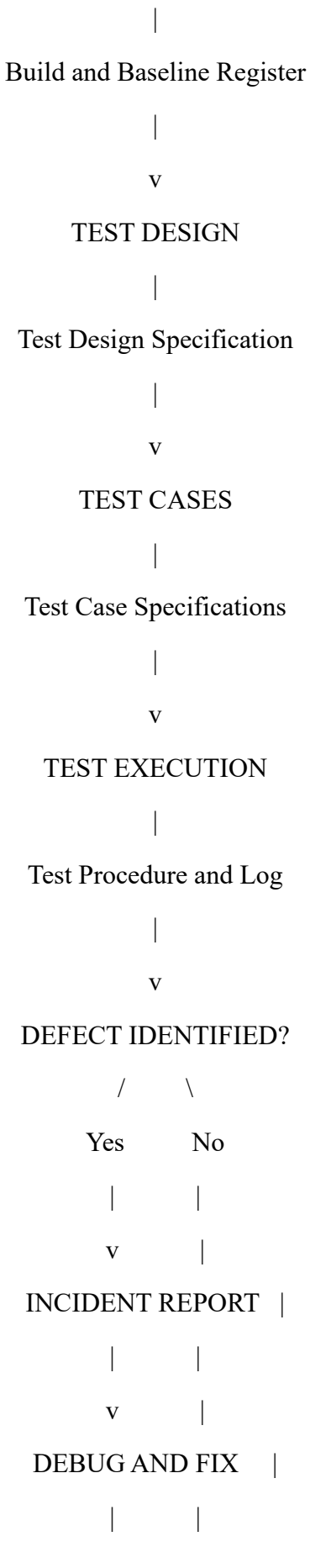
|

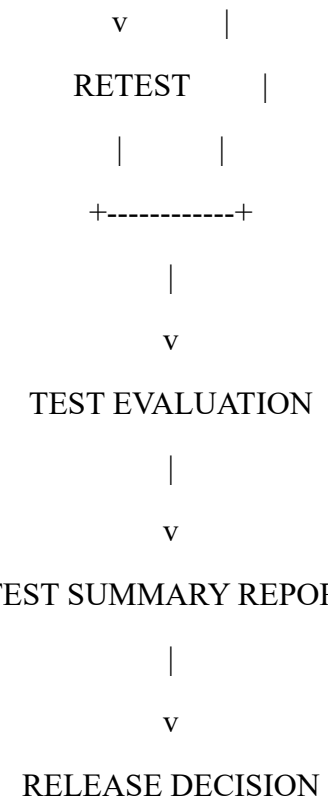
Test Plan Document

|

v

IDENTIFY TEST ITEMS


Build and Baseline Register
TEST DESIGN
Test Design Specification
TEST CASES
Test Case Specifications
TEST EXECUTION
Test Procedure and Log
DEFECT IDENTIFIED?
Yes
No
INCIDENT REPORT
DEBUG AND FIX



9. Document Ownership

Table 43: Document responsibility

Document	**Owner**
Test plan	Test manager
Test design specification	Test lead/tester
Test case specification	Testers
Test procedure	Testers
Test log	Test executor
Test incident report	Tester who identifies the defect
Test summary report	Test manager
Build baseline register	Build engineer
Requirement traceability matrix	Business analyst and test team

10. Comparison of the Three Documentation Sets

Table 44: Evaluation

Criterion	Full IEEE set	Lean tool-generated	Risk-tailored hybrid
Completeness	Very high	Medium	High
Auditability	Very high	High if configured	Very high for critical modules
Effort	High	Low	Medium
Readability	High when maintained	Depends on tool	High
Ease of change	Medium	Very high	High
Financial risk coverage	High	Depends on configuration	Very high

11. Pilot-Year Problems Exposed by Each Set

Pilot problem	Full IEEE set	Lean tool-generated	Hybrid set
Wrong bill at 500 units	Detailed test cases expose it	Automated test result exposes it	Full billing documents expose it
Duplicate payment	Incident and test records	Tool records repeated transaction	Full payment documentation
Wrong released build	Plan and item identification	Depends on tool configuration	Baseline and transmittal records
Unknown testing progress	Test logs and summary	Dashboard	Dashboard plus summary

12. Rule for Skipping a Document

A document may be shortened or skipped only when:

- The feature is low risk.

- The required information is already captured in an approved tool.
- No legal or audit requirement demands a separate document.
- The test manager approves the decision.
- The information remains accessible and traceable.

Documents relating to critical billing and payment functions must not be skipped.

13. Can AI-Generated Documents Be Accepted Without Review?

AI-generated documents may help reduce preparation effort, but they should not be accepted for audit without human review.

The reviewer must verify:

- Correct requirement references.
- Correct expected values.
- Complete test coverage.
- Correct build identifiers.
- Accurate defect details.
- No invented execution results.
- Approval and ownership information.

14. Final Recommendation

I recommend **Approach C – Risk-Tailored Hybrid Documentation**.

It provides detailed evidence for billing and payment while reducing unnecessary paperwork for lower-risk modules. This is suitable for the Corporation because billing accuracy and payment reliability have a direct impact on consumers.

Conclusion

Documentation helps maintain testing evidence and supports release decisions. Full IEEE-style documents provide completeness, tool-generated documents reduce effort, and a hybrid approach balances both. PowerPay should use full documentation for high-risk modules and lean records for supporting features.

TASK 9 – Re-evaluate and Redesign the Test Process

Changed Requirement

PowerPay is now selected for the entire state and must support 12 lakh consumers. The system must also process smart-meter readings every 15 minutes. The testing period is reduced from 10 weeks to 4 weeks.

The team is now divided between two offices, one tester is withdrawn, and a data-protection law requires consumer test data to be masked and every test result to be auditable.

The original process must therefore be reviewed and redesigned to satisfy the new conditions.

Prediction Before the Re-evaluation

Before evaluating the new requirements, I expected the original process to become insufficient because the testing period and team capacity have decreased while the system size and testing requirements have increased.

I expected the modified process to improve communication and speed, while the redesigned central test process would provide stronger audit control and evidence capture.

TASK 9.1 – Impact Analysis

1. Changed Project Conditions

Table 45: Original and revised project conditions

Parameter	**Original condition**	**Changed condition**
Consumers	3 lakh	12 lakh
Smart-meter upload	Not included	Every 15 minutes
Testing duration	10 weeks	4 weeks
Testers	4	3
Team location	One location	Two offices
Test conditions	180	Approximately 400
Test data	Normal test data	Masked consumer data

Audit evidence	Standard records	Every result must be auditable
Reporting	Periodic reporting	Daily evidence-based reporting

2. Impact of Each Change

Table 46: Impact analysis

Change	**Impact on testing**
Increase to 12 lakh consumers	Higher load and scalability testing required
Smart-meter uploads every 15 minutes	Additional data processing and scheduling tests required
Testing reduced to 4 weeks	More prioritisation and parallel work required
One tester withdrawn	Reduced manual testing capacity
Two office locations	Stronger coordination and shared tools required
Masked test data	Test data preparation and validation required
Auditable results	Evidence capture and access control required

3. Consequences of Using the Original Plan Unchanged

If the original plan is used without modification:

- Testing may not finish within four weeks.
- Performance risks may remain undetected.
- Smart-meter upload defects may escape.
- The smaller team may become overloaded.
- Different offices may use inconsistent test records.
- Unmasked data may violate the data-protection requirement.
- Incomplete evidence may create audit problems.

Therefore, the original plan cannot be used unchanged.

4. Revised Effort Calculation

Given:

- Original test conditions = 180.
- Revised test conditions = 400.
- Design effort per condition = 0.4 hour.
- Execution effort per cycle = 0.25 hour.
- Execution cycles = 3.
- Retest effort = 20%.
- Management overhead = 15%.

Step 1: Test design effort

400 × 0.4 = **160 hours**

Step 2: Execution effort

400 × 0.25 × 3 = **300 hours**

Step 3: Base effort

160 + 300 = **460 hours**

Step 4: Retest effort

460 × 20 ÷ 100 = **92 hours**

Step 5: Subtotal

460 + 92 = **552 hours**

Step 6: Management overhead

552 × 15 ÷ 100 = **82.8 hours**

Total required effort

552 + 82.8 = **634.8 person-hours**

Therefore, the revised testing requirement is approximately **635 person-hours**.

5. Available Capacity

Available testers = 3

Testing duration = 4 weeks

Working hours per tester per week = 30

Available capacity:

3 × 4 × 30 = **360 person-hours**

Table 47: Revised effort feasibility

Description	**Person-hours**
Required effort	634.8
Available capacity	360
Shortfall	274.8

Shortfall:

634.8 − 360 = **274.8 person-hours**

The required testing effort is greater than the available tester capacity.

Available capacity percentage:

360 ÷ 634.8 × 100 = **56.71%**

The team can provide only approximately 56.71% of the estimated effort if all other assumptions remain unchanged.

This indicates that the original full testing scope is not feasible with the reduced team and schedule.

6. Data Masking Requirements

The new data-protection rule requires consumer data used in testing to be masked or removed where necessary.

Table 48: Test data protection

Data field	Original example	Masked example
Consumer name	Actual consumer name	Consumer_001
Consumer ID	Actual ID	TEST-C-001
Mobile number	Actual number	9XXXXXX001
Address	Actual address	Address_Masked
Meter number	Actual meter number	MTR-TEST-001
Payment reference	Actual reference	PAY-TEST-001
Account details	Actual account information	Removed or tokenised

7. Information to Remove from Reports

The following information must not appear in general test reports:

- Full consumer names.
- Full mobile numbers.
- Actual residential addresses.
- Real payment credentials.
- Bank account details.
- Unnecessary personal identifiers.

Reports should use test identifiers instead of real consumer details.

8. Audit Evidence Requirements

Each test result should record:

- Unique test case ID.
- Requirement ID.
- Build identifier.
- Test environment.
- Test data reference.

- Execution date and time.
- Tester identity or approved user identifier.
- Expected result.
- Actual result.
- Pass/fail status.
- Defect reference, if applicable.
- Evidence attachment or location.

Conclusion for 9.1

The revised requirements increase the testing effort and introduce stricter data-protection and audit responsibilities. The calculated shortfall of 274.8 person-hours shows that the original process is not feasible without changes to scope, resources or automation.

TASK 9.2 – Evaluation of the Original Process

1. Original Process Review

The original process was designed for 10 weeks, four testers and approximately 180 test conditions. The changed project has four weeks, three testers and approximately 400 test conditions.

The original process must be checked against these conditions.

2. Original Process Validity Checklist

Table 49: Evaluation checklist

Original process element	**Still valid?**	**Reason**
Risk-based testing	Yes	Critical features still require higher priority
Requirement traceability	Yes	Required for audit and coverage
Baseline identification	Yes	Build control remains essential

Systematic debugging	Yes	Defects must still be located and corrected
Four-person tester allocation	No	Only three testers remain
Ten-week schedule	No	New schedule is four weeks
Original 180 test conditions	No	Scope has increased to about 400
Original test server assumptions	Must be reviewed	Load and data capacity have increased
Existing reporting frequency	Must be changed	Daily audit-ready reporting is required
Existing test data approach	No	Consumer data must be masked

3. Effect of Original Pass/Fail Criteria

The original pass/fail criteria may not be sufficient because the new project has a larger user population and more critical functions.

A high pass percentage does not guarantee readiness if the remaining failed test cases relate to:

- Incorrect billing.
- Duplicate payments.
- Smart-meter upload failures.
- Data protection.
- Performance under high load.

Therefore, critical failures must be treated separately from general pass percentages.

4. Revised Exit Criteria

The following exit criteria are proposed:

1. No open critical billing defects.
2. No open critical payment defects.
3. All mandatory billing boundary tests must pass.

4. Duplicate-payment prevention tests must pass.
5. Smart-meter upload tests must satisfy approved processing requirements.
6. All executed tests must have auditable records.
7. All consumer test data must be masked.
8. Tested and released build identifiers must match.
9. Remaining non-critical defects must be documented and formally accepted by the authorised representative.

5. Least Reliable Original Reporting Measure

The least reliable measure under the new schedule would be the overall pass percentage when used alone.

For example, a 95% pass rate may appear positive, but the remaining failures could include a critical duplicate-payment defect.

Therefore, daily reports must show pass percentage together with severity, open critical defects, execution coverage and blocked tests.

6. Parts of the Original Process That Remain Useful

The following practices remain useful:

- Requirement traceability.
- Baseline identification.
- Risk-based testing.
- Systematic debugging.
- Defect reporting.
- Regression testing.
- Test summary reports.

However, they must be supported by stronger automation, data masking, shared tools and more frequent reporting.

Conclusion for 9.2

The original testing principles remain useful, but the original schedule, staffing assumptions and reporting arrangements are no longer suitable. The process must be adapted to handle higher risk, reduced capacity and stronger audit requirements.

TASK 9.3 – Compare and Adapt

1. Approach A – Retain the Process and Cut Scope by Risk

This approach retains the original plan, documents and team structure but reduces the number of features tested according to risk.

Advantages

- Quick to implement.
- Requires fewer process changes.
- Maintains familiar documentation.
- Focuses on high-risk features.

Limitations

- The original structure may not support two offices.
- Audit evidence may remain insufficient.
- Reduced scope may leave smart-meter risks uncovered.
- The team may still face capacity problems.

2. Approach B – Modify the Process

This approach introduces:

- Short iteration-level test plans.
- Testers embedded with module teams.
- Shared test-management tools.
- Live progress reporting.
- Daily coordination between offices.

Advantages

- Supports parallel testing.
- Improves communication.
- Provides faster feedback.
- Helps manage changing priorities.

Limitations

- Requires common working procedures.
- Tool setup and training are necessary.
- Audit evidence must be configured correctly.

3. Approach C – Redesign the Organisation and Policy

This approach introduces:

- A central test group.
- A revised testing policy.
- Lightweight standard documents.
- Tool-generated reports.
- Automated evidence capture.
- Mandatory data masking.
- Centralised release criteria.

Advantages

- Strong audit control.
- Standardised processes across both offices.
- Better management visibility.
- Reduces manual documentation.
- Supports central risk prioritisation.

Limitations

- Higher initial setup effort.
- Requires management support.
- May reduce local flexibility.
- Depends on reliable shared tools.

4. Comparison of the Three Strategies

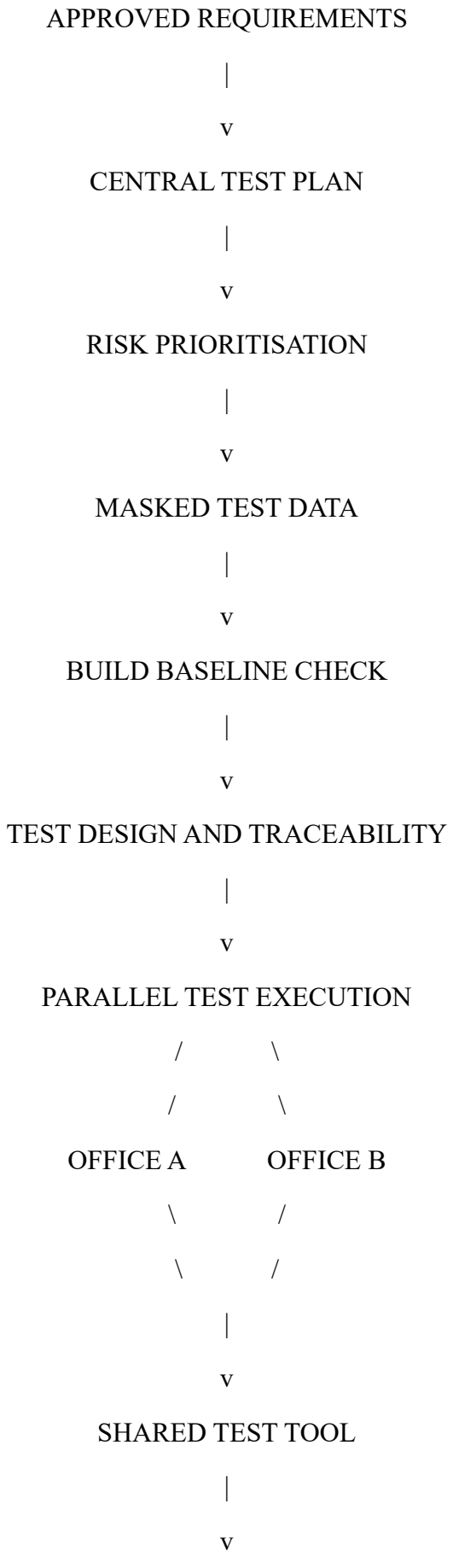
Table 50: Strategy comparison

Criterion	A. Retain and cut scope	B. Modify process	C. Redesign
Schedule feasibility	Medium	High	High after setup
Risk coverage	Medium	High	Very high
Audit evidence	Medium	High	Very high
Cost	Low	Medium	Medium to high
Team effectiveness	Medium	High	High
Data protection	Requires changes	Can be integrated	Built into process
Two-office coordination	Limited	Very good	Very good
Automation	Existing level	Increased	Strongly supported

5. Revised Test Process

I recommend a redesigned process based on Approach C, while using the iterative coordination practices of Approach B.

Revised flowchart


APPROVED REQUIREMENTS
CENTRAL TEST PLAN
RISK PRIORITISATION
MASKED TEST DATA
BUILD BASELINE CHECK
TEST DESIGN AND TRACEABILITY
PARALLEL TEST EXECUTION
OFFICE A
OFFICE B
SHARED TEST TOOL

AUTOMATED REPORTING

|

v

DEFECT TRIAGE AND FIX

|

v

REGRESSION

|

v

EXIT CRITERIA REVIEW

|

v

RELEASE DECISION

6. Revised Test Plan Outline

Table 51: Revised plan

Component	**Revised arrangement**
Scope	Critical billing, payment, smart-meter and consumer functions
Schedule	Four weeks
Team	Three testers and supporting project members
Organisation	Central coordination with work across both offices
Test approach	Risk-based and iterative
Test data	Masked or synthetic
Configuration	Approved build baseline
Test records	Shared test-management tool
Reporting	Daily dashboard and weekly summary

Audit	Automatic evidence capture
Release	Formal approval against revised exit criteria

7. Revised Test Priorities

Because the available effort is limited, the team must prioritise features according to risk.

Table 52: Revised test priority

Priority	**Feature**	**Reason**
P1 – Critical	Billing slab calculation	Direct financial impact
P1 – Critical	Online payment	Duplicate or incorrect payment risk
P1 – Critical	Smart-meter upload	High frequency of incoming data
P1 – Critical	Consumer data protection	Legal and privacy requirement
P2 – High	Meter-reading validation	Incorrect data affects bills
P2 – High	Login and account access	Consumer access and security
P2 – High	Load and scalability	Increased user population
P3 – Medium	Complaints and outage reporting	Consumer service continuity
P3 – Medium	SMS notifications	Supporting service

Lower-priority features should still receive appropriate testing. They must not be silently excluded; any scope reduction must be documented and approved.

8. Daily Reporting Set for the Chief Engineer

Table 53: Daily reporting metrics

Metric	**Purpose**
Planned test cases	Shows total scope
Executed test cases	Shows daily progress

Passed test cases	Shows successful execution
Failed test cases	Identifies current problems
Blocked test cases	Shows environment or dependency issues
Open critical defects	Highlights release risk
Open major defects	Shows remaining important issues
Billing boundary status	Protects tariff calculations
Duplicate-payment status	Protects payment correctness
Smart-meter upload status	Monitors new functionality
Masked-data compliance	Confirms data protection
Build identifier	Confirms tested version
Evidence completion	Supports audit

9. Revised Release Exit Criteria

The system can be considered for release only when:

- No critical billing defect remains open.
- No critical payment defect remains open.
- The 500-unit billing boundary passes.
- All approved tariff boundaries pass.
- Duplicate payment is prevented.
- Smart-meter uploads are processed according to the approved requirements.
- Required security and data-protection checks pass.
- All mandatory executed test results are auditable.
- All test data used follows masking rules.
- The final build matches the tested baseline.

- The authorised Corporation representative approves the release.

Conclusion for 9.3

The redesigned approach provides stronger central control, audit evidence and risk coverage. The combination of a central process with iterative team coordination is more suitable for the revised four-week schedule.

TASK 9.4 – Evidence-Based Defence

1. Defence of the Selected Strategy

I recommend **Approach C – Redesign the Organisation and Policy**, supported by selected practices from Approach B.

The decision is based on the revised project conditions and the calculations.

Table 54: Evidence supporting the decision

Evidence	**Observation**	**Decision impact**
Consumer population	Increased from 3 lakh to 12 lakh	More scalability testing
Test conditions	Increased from 180 to 400	More test design and execution
Schedule	Reduced from 10 weeks to 4 weeks	Need for prioritisation
Testers	Reduced from 4 to 3	Reduced capacity
Effort requirement	634.8 hours	High workload
Available capacity	360 hours	274.8-hour shortfall
Data protection	Masking required	Revised data process
Audit	Every result must be auditable	Automated evidence capture

2. What the Redesigned Process Gives Up

The redesigned process may reduce some flexibility and completeness because of the shorter schedule.

Possible compromises include:

- Lower-risk test scenarios may receive less execution time.
- Some detailed narrative documents may be replaced by tool-generated records.
- Less time may be available for exploratory testing.
- Manual regression may be reduced in favour of automation.
- Some non-critical improvements may be postponed.

These compromises must be documented and approved.

3. Condition Under Which the Redesigned Process May Fail

The process may fail if the regulator introduces a major tariff change near the end of the four-week testing period.

Such a change may affect:

- Billing calculations.
- Expected test results.
- Test data.
- Regression tests.
- Release approval.
- The testing schedule.

If this happens, the team must reassess the affected risks, update the tariff requirements, prepare new test cases and repeat the critical billing regression tests.

The release should be delayed if the change creates an unresolved critical billing risk.

4. Final Decision Table

Table 55: Final decision

Decision	**Justification**	**What if not selected?**
Adopt a central test process	Provides consistent control across both offices	Different teams may follow different processes

Use risk-based prioritisation	Focuses on billing and payment risks	Critical functions may receive insufficient attention
Use masked test data	Supports data protection	Consumer information may be exposed
Use shared test-management tools	Improves coordination and traceability	Reporting may become inconsistent
Capture audit evidence automatically	Supports audit requirements	Test results may be difficult to verify
Retain formal release approval	Prevents unsupported release decisions	Critical defects may reach consumers

5. Overall Conclusion

The revised PowerPay project has a larger user population, additional smart-meter functionality, a shorter testing period and reduced tester availability. The original testing process cannot meet these requirements without modification.

The effort calculation shows a requirement of approximately 634.8 person-hours against an available capacity of 360 person-hours, creating a shortfall of 274.8 hours.

Therefore, the Corporation should redesign its testing organisation and policy, introduce risk-based priorities, use shared test-management tools, enforce data masking and capture auditable evidence automatically.

The redesigned process provides a stronger foundation for the revised project. However, release approval must depend on critical billing and payment results rather than on schedule completion or overall pass percentage alone.

OVERALL DISCUSSION AND ANALYSIS

The completion of all nine tasks provided a clear understanding of the different activities involved in software testing and quality assurance. The electricity billing system was considered as the main application throughout the assignment. Each task focused on an important part of the testing process, starting from the development of a testing prototype and continuing through effort estimation, requirement traceability, test case preparation, testing policies, team organization, skill allocation, documentation, and test planning.

The first task helped in understanding how a billing system can be tested by identifying possible faults in its functions. Since billing applications deal with customer information, electricity consumption, tariff calculations, and payment details, even a small error can affect the final bill. Therefore, different test conditions were considered to check whether the system produces correct results. Fault injection also helped demonstrate how defects can be identified through testing.

The second and ninth tasks focused on testing effort estimation and resource planning. These tasks showed that testing requires proper calculation of time, workload, and available resources. Estimating the effort before starting the testing process helps the team understand whether the work can be completed within the given schedule. It also helps identify situations where additional resources or changes in the testing schedule may be required.

The traceability matrix prepared in Task 3 established a connection between software requirements and their corresponding test cases. This is important because every requirement must be verified during testing. Without traceability, some requirements may be missed, resulting in incomplete testing. The test cases and incident reports prepared in Task 4 provided a structured way to verify the expected behaviour of the billing system and record problems found during testing.

Task 5 explained the importance of defining a testing policy, selecting suitable metrics, and establishing exit criteria. These elements provide a common direction for the testing team. Task 6 focused on organizing the testing team and assigning responsibilities to different members. Task 7 considered the skills and experience required for different testing roles. These activities showed that successful testing depends not only on tools and test cases but also on proper teamwork and communication.

The documentation and deliverables considered in Task 8 highlighted the importance of maintaining testing records. Documents such as test plans, test cases, defect reports, execution reports, and summary reports help track the progress of testing activities. They also provide useful information for future maintenance and improvement.

Overall, the tasks demonstrated that software testing is a systematic process. Each activity is connected to the others, and proper planning at every stage helps improve the quality and reliability of the final application.

LEARNING OUTCOMES

By completing this assignment, several important concepts and practical skills related to software testing were understood.

1. **Understanding Software Testing:** Learned how testing is used to identify defects and verify whether an application meets its specified requirements.
2. **Test Case Preparation:** Understood how to prepare test cases using test data, preconditions, execution steps, expected results, and actual results.
3. **Fault Identification:** Learned how faults can be introduced into a sample application and how test cases can help identify incorrect behaviour.
4. **Effort Estimation:** Gained knowledge of calculating testing effort based on requirements, execution cycles, retesting, and management overhead.
5. **Requirement Traceability:** Understood how a traceability matrix connects requirements with test cases and helps ensure complete requirement coverage.
6. **Defect Reporting:** Learned how to record defects with appropriate descriptions, severity, priority, and supporting information.
7. **Testing Policy and Metrics:** Understood the importance of defining testing guidelines and measuring progress through suitable testing metrics.
8. **Team Management:** Learned how testing responsibilities can be divided among team members according to their roles, skills, and experience.
9. **Testing Documentation:** Understood the importance of maintaining proper records throughout the testing process.

10. **Test Planning:** Gained knowledge of preparing a testing schedule based on available time, team size, and estimated workload.

These learning outcomes helped develop a better understanding of how software testing activities are planned, organized, performed, and evaluated in a real-world software development environment.

RECOMMENDATIONS FOR IMPROVING THE BILLING SYSTEM

Based on the testing activities carried out in this assignment, a few recommendations can be considered to improve the reliability and performance of the electricity billing system.

1. Perform Regular Testing

The billing system should be tested regularly whenever new features or changes are introduced. Regular testing helps identify defects at an early stage and reduces the possibility of errors reaching the users.

2. Give Importance to Boundary Value Testing

Special attention should be given to electricity consumption values near tariff boundaries. Values such as 99, 100, 101, 299, 300, 301, 499, 500, and 501 should be checked carefully to verify that the correct tariff is applied.

3. Maintain Accurate Customer Information

Customer details, meter readings, billing history, and payment records should be maintained accurately. Validation checks should be included to prevent incomplete or incorrect information from being stored.

4. Improve Defect Tracking

Every identified defect should be recorded and monitored until it is resolved. The testing team should maintain proper defect reports so that developers can understand the issue and take corrective action.

5. Maintain Proper Testing Documentation

All test plans, test cases, test results, and defect reports should be stored systematically. Proper documentation makes it easier to review previous testing activities and understand the changes made to the application.

6. Allocate Resources Effectively

Testing tasks should be assigned according to the skills and availability of team members. Proper workload distribution helps avoid delays and improves the efficiency of the testing process.

7. Conduct Regression Testing

Whenever a defect is corrected, the related test cases should be executed again. Regression testing should also be performed to ensure that the correction has not affected other existing functions.

FUTURE ENHANCEMENTS

The electricity billing system can be improved further by introducing additional features and stronger testing practices.

- **Automated Testing:** Repeated test cases can be automated to reduce manual effort and improve testing speed.
- **Online Payment Verification:** Payment-related functions can be tested to ensure that transactions are recorded correctly and duplicate payments are avoided.
- **Improved Security Testing:** Customer information and payment details should be protected through suitable security controls.
- **Performance Testing:** The system can be tested with a large number of customers and billing records to evaluate its response time and stability.
- **User Interface Testing:** Different screens, input fields, buttons, and messages can be checked to ensure that customers can use the application easily.
- **Database Validation:** Database records can be verified to ensure that customer details, meter readings, and payment information remain consistent.
- **Automated Defect Reporting:** Defect tracking tools can be used to record, assign, and monitor defects more efficiently.
- **Continuous Testing:** Testing can be included throughout the software development process so that problems are identified as early as possible.

These improvements can help make the billing application more reliable, secure, user-friendly, and suitable for handling customer billing activities.

FINAL CONCLUSION

This assignment provided practical knowledge of software testing and the different activities involved in ensuring software quality. By using an electricity billing system as the main example, the assignment covered important areas such as prototype development, fault identification, testing effort estimation, requirement traceability, test case preparation, defect reporting, testing policies, team organization, skill allocation, documentation, and test planning.

The activities showed that software testing is not limited to finding errors after an application has been developed. It begins with understanding the requirements and continues through planning, designing test cases, executing tests, recording defects, retesting corrected functions, and preparing final reports. Each stage plays an important role in improving the quality of the application.

The assignment also highlighted the importance of proper resource management and communication among team members. A well-prepared testing plan helps the team complete its work within the available time and resources. Similarly, maintaining accurate documentation makes the testing process easier to monitor and review.

In the case of the electricity billing system, accurate calculation of electricity charges, correct customer details, reliable payment processing, and proper storage of billing records are essential. Testing these functions helps reduce errors and improves the confidence of users in the application.

Finally, the completion of this assignment improved the understanding of software testing concepts and their practical applications. It provided knowledge of how testing activities are organized in a systematic manner and how different testing techniques contribute to software quality. The concepts learned through these tasks can be applied to other software applications in future academic projects and professional environments.